

A1. 轻量 RTL 仿真器

题目描述

设计并实现一个轻量级事件驱动的 Verilog RTL 仿真器，支持基本语法解析与时序仿真。

1. 核心要求

1.1 语言解析

支持 Verilog-2001 核心语法子集：

- module / endmodule 层级实例化
- always @(*) / always @(posedge clk)
- assign 连续赋值
- reg / wire 类型声明
- integer 变量
- if-else / case / for 过程语句
- 参数化 parameter
- `define / `include 预处理

1.2 仿真引擎

实现事件驱动调度器：

- 事件队列：Active -> NBA 两阶段调度
- 非阻塞赋值支持
- Delta cycle 控制

其他要求：

- 时序元件建模：DFF / 带异步复位的寄存器
- 内置系统函数：\$display / \$readmemh / \$time / \$finish / \$dumpvars / \$fdisplay / \$fgets / \$fopen

1.4 增量编译

- 文件级依赖分析，修改单个文件后仅重新编译受影响模块

1.5 并发仿真

- 支持多线程仿真加速（利用 CPU 多核）
- 模块间事件同步

2. 交付物

- 源代码
- Makefile测试接口

3. 测试接口规范

参赛者需要在提交目录中提供统一的 **Makefile** 作为评测入口。评测系统只调用约定的 make target，不依赖参赛者内部工程结构。

3.1 Makefile 必选目标

参赛者提交的 Makefile 必须至少支持以下目标：

```
build
compile_sim
run
parallel_run
```

各目标含义如下：

目标	作用
build	编译参赛者提交的仿真器源码，生成仿真器可执行文件
compile_sim	读取评测方提供的 filelist，解析并编译 RTL 与 testbench，生成可运行的仿真产物
run	执行默认单线程仿真，生成 tb/output.mem
parallel_run	按指定线程数执行并行仿真，生成 tb/output.mem

3.2 Makefile 参数

评测系统通过 make 变量传入测试配置。参赛者的 Makefile 应支持以下变量：

变量	含义	默认建议
FILELIST	Verilog 源文件列表路径	filelist.txt
TOP	仿真顶层模块名	tb
THREADS	并行仿真线程数	4
SIM_EXE	compile_sim 生成的仿真可执行文件或运行入口	sim.out

推荐默认值：

```
FILELIST ?= filelist.txt
TOP ?= tb
THREADS ?= 4
SIM_EXE ?= sim.out
```

3.3 评测调用流程

```
make build
```

普通单线程功能测试按以下顺序调用：

```
make compile_sim FILELIST=filelist.txt TOP=tb
make run
diff -u tb/output_ref.mem tb/output.mem
```

并行仿真能力测试按以下方式调用：

```
make compile_sim FILELIST=filelist.txt TOP=tb
make parallel_run THREADS=4
diff -u tb/output_ref.mem tb/output.mem
```

增量式编译能力测试单独按以下方式调用：

```
make compile_sim FILELIST=filelist_bug.txt TOP=tb
make compile_sim FILELIST=filelist.txt TOP=tb
make run
diff -u tb/output_ref.mem tb/output.mem
```

filelist_bug.txt 只用于增量式编译能力测试，不用于普通单线程功能测试和并行仿真能力测试。

3.4 输出要求

- build 成功后，应生成仿真器本体或可被后续目标调用的构建产物。
- 每次 compile_sim 成功后，应生成或更新 run / parallel_run 可直接执行的仿真产物。
- run 和 parallel_run 必须在当前测试 case 目录下生成 tb/output.mem。

3.5 Golden 对比规则

仿真器执行 tb.v，由 tb.v 读取 tb/input.mem 并写出 tb/output.mem。最终正确性由评测系统在仿真器外部执行：

```
diff -u tb/output_ref.mem tb/output.mem
```

判定规则：

- make build 失败：构建失败
- 任意一次 make compile_sim 失败：编译仿真工程失败
- make run 或 make parallel_run 失败：仿真失败
- 未生成 tb/output.mem：仿真失败
- diff 存在差异：功能错误

- diff 无差异：该测试 case 通过

3.6 参考 Makefile 框架

以下仅为接口示例，参赛者可根据自身实现替换具体命令：

```
FILELIST ?= filelist.txt
TOP ?= tb
THREADS ?= 4
SIM_EXE ?= sim.out

.PHONY: build compile_sim run parallel_run clean

build:
    # Compile simulator source code and generate simulator executable.
    # Example: g++ -O2 -o simulator src/*.cc

compile_sim:
    # Use simulator to read $(FILELIST), elaborate $(TOP), and generate $(SIM_EXE).
    # Example: ./simulator --compile -f $(FILELIST) --top $(TOP) -o $(SIM_EXE)

run:
    # Run single-thread simulation and generate tb/output.mem.
    # Example: ./$(SIM_EXE)

parallel_run:
    # Run parallel simulation with $(THREADS) threads and generate tb/output.mem.
    # Example: ./$(SIM_EXE) --threads $(THREADS)

clean:
    rm -f simulator $(SIM_EXE) tb/output.mem
```

4. 测试Case 权重总表

case	建议	F1 语言解析	F2 仿真正确性	P1 编译性能	P2 基础仿真性能	P3 多核加速比	总分
basic01	公开	1	1	0	0	0	2
basic02	公开	1	1	0	0	0	2
basic03	公开	1	1	0	0	0	2
basic04	公开	1	1	0	0	0	2
basic05	公开	1	1	0	0	0	2
alu	公开	1	1	0	0	0	2
priority_encoder	公开	1	1	0	0	0	2
i2c	公开	1	2	0	0	0	3
ip	公开	1	1	0	0	0	2
axis_fifo	公开	1	1	0	0	0	2
sha256	公开	1	1	0	0	0	2
GEMM	公开	1	1	2	3	4	11

4.1 性能 baseline 计分规则

性能分全部相对于赛题方 baseline 计算。赛题方 baseline 对每个性能 case 提供以下数据：

```
T_compile_baseline = baseline 编译时间
T1_baseline        = baseline 单核仿真时间
T4_baseline        = baseline 4 核仿真时间
S_baseline         = T1_baseline / T4_baseline
```

参赛者提交在同一 case 上得到：

```
T_compile_submit = 参赛工具编译时间
T1_submit        = 参赛工具单核仿真时间
T4_submit        = 参赛工具 4 核仿真时间
```

注: 编译时间使用第一次编译时间和增量式编译时间的平均值。功能正确性是性能计分前置条件：对应 case 的 F1/F2 不通过时，该 case 的 P1/P2/P3 均记 0。

P1/P2 时间类性能

P1 编译性能和 P2 基础仿真性能使用同一个时间得分函数。

P1：

```
R_compile = T_compile_baseline / T_compile_submit
Q_P1 = TimeScore(R_compile)
```

P2：

```
R_sim = T1_baseline / T1_submit
Q_P2 = TimeScore(R_sim)
```

TimeScore(R) 定义：

R	含义	得分系数
$R < 0.5$	比 baseline 慢超过 2x	0
$0.5 \leq R < 1.0$	慢于 baseline	$1.2 * (R - 0.5)$
$R = 1.0$	与 baseline 持平	0.6
$1.0 < R < 2.0$	快于 baseline	$0.6 + 0.4 * (R - 1.0)$
$R \geq 2.0$	比 baseline 快 2x 或以上	1.0

因此，达到 baseline 得到 60% 的该项性能分；超过 baseline 会继续加分；达到 baseline 2x 或以上拿满分。

P3 多核加速比

P3 比较参赛工具相对于 baseline 的 4 核有效加速比。

```
S_submit_raw = T1_submit / T4_submit
S_submit_cap = T1_baseline / T4_submit
S_submit_eff = min(S_submit_raw, S_submit_cap)
R_multi = S_submit_eff / S_baseline
Q_P3 = RatioScore(R_multi)
```

RatioScore(R) 定义：

R	含义	得分系数
$R < 0.5$	明显低于 baseline	0
$0.5 \leq R < 1.0$	低于 baseline	$1.2 * (R - 0.5)$
$R = 1.0$	与 baseline 持平	0.6
$1.0 < R < 1.5$	优于 baseline	$0.6 + 0.8 * (R - 1.0)$
$R \geq 1.5$	达到 baseline 的 1.5x 或以上	1.0

P3 额外约束：

- 1. 4 核仿真输出必须与单核仿真逐周期一致，否则该 case 的 $Q_{P3} = 0$ 。
- 2. 如果同一 case 的 $Q_{P2} < 0.3$ ，则该 case 的 $Q_{P3} = 0$ 。
- 3. 如果同一 case 的 $Q_{P2} < 0.6$ ，则该 case 的 Q_{P3} 最高为 0.6。

性能得分计算

每个性能 case 的最终性能得分为：

```
PerfScore(case) =
    W_P1(case) * Q_P1(case) +
    W_P2(case) * Q_P2(case) +
    W_P3(case) * Q_P3(case)
```

其中 $W_{P1}/W_{P2}/W_{P3}$ 来自 Case 权重总表。非性能 case 的 P1/P2/P3 权重为 0，不参与性能计分。